Ikaskuntza Birtual eta Digitalizatuen LHII
CIFP de Aprendizajes Virtuales y Digitalizados

Página para cálculo de costos de viaje

Raúl Vázquez Iglesias

Desarrollo de Aplicaciones Web
Desarrollo Web en Entorno Cliente

Curso: 2026/27

EUSKO JAURLARITZA GOBIERNO VASCO
HEZKUNTZA SAILA
Lanbide Heziketako Sailburuordetza
DEPARTAMENTO DE EDUCACIÓN
Viceconsejería de Formación Profesional

ÍNDICE

1. ACLARACIONES 1
2. ENLACES 1
3. CAPTURAS DE PANTALLA 2
4. AUTOEVALUACIÓN 6
5. BIBLIOGRAFÍA 6

1. ACLARACIONES

Responde a las siguientes preguntas:

- ¿Tuviste dificultades al realizar la tarea?

  Estoy aprendiendo y hay muchas cosas nuevas. Sin embargo, mi principal problema ha sido con GitHub. Cuando era programador trabajaba con Subversion y, por aquel entonces, GitHub aún no era tan popular como lo es ahora. Me he liado varias veces con esto y he tenido que acudir a Internet para tratar de solventarlo.
- ¿Faltan recursos? como vídeos, materiales o tareas de aprendizaje
  El material online es de primera calidad y estoy aprendiendo mucho. Me falta tiempo para poder sumergirme completamente en la materia, pero por parte del docente, no puede estar más completo. Además, el hecho de que la tarea fuera troceada en ejercicios me ha ayudado a aislar partes y ver el problema por separado.
- ¿Tienes alguna propuesta de mejora concreta para esta UD?
  Al ser una UD inicial, eché de menos que las tareas iniciales fueran más simples, aunque más numerosas hasta alcanzar el nivel de la tarea final.

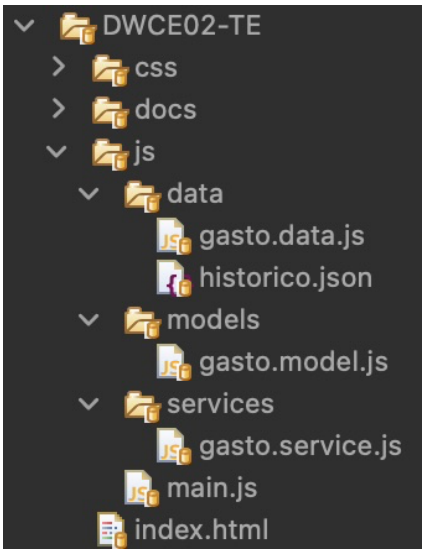
2. ENLACES

- Repositorio de Código: https://github.com/RaulVazquezEduDaw/mi-primer-repo-dwec/tree/main/DWCE02-TE
- Aplicación desplegada: la he desplegado en local.
- Vídeo-demostración: https://youtu.be/iCYMU3WZm88

3. CAPTURAS DE PANTALLA

Ejercicio 1: Estructura de directorios y enlazado de ficheros

- Creación de la estructura de directorios:



- Enlace de los ficheros main.js y style.css en el <head> del index.html:

```
<script type="module" src="js/main.js"></script>
<link rel="stylesheet" href="css/style.css">
```

Ejercicio 2: Acceso a datos

- **Creación del modelo de datos GastoCombustible (gasto.model.js):**

```
export class GastoCombustible {
    constructor(id, vehicleType, date, kilometers, precioViaje) {
        this.id = id;
        this.vehicleType = vehicleType;
        this.date = new Date(date);
        this.kilometers = parseFloat(kilometers);
        this.precioViaje = parseFloat(precioViaje);
    }
}
```

- **Copia de los datos del JSON (historico.json) y parseo de cada registro en un objeto de tipo GastoCombustible (GASTOS_DB) (gasto.data.js):**

```
// Importo la clase GastoCombustible desde su modelo
import { GastoCombustible } from '../models/gasto.model.js';

/**
 * Copia el contenido de historico.json dentro de la constante jsonHistorico y procesa ese contenido como
 * JSON para obtener los registros del histórico.
 */
const jsonHistorico = `[
  {"id":1,"vehicleType":"moto","date":"2015-04-04T00:00:00.000Z","kilometers":28,"precioViaje":1.68},
  {"id":2,"vehicleType":"furgoneta","date":"2015-09-18T00:00:00.000Z","kilometers":145,"precioViaje":14.10},
  {"id":3,"vehicleType":"camion","date":"2015-10-30T00:00:00.000Z","kilometers":350,"precioViaje":56.00},
  {"id":4,"vehicleType":"moto","date":"2016-05-17T00:00:00.000Z","kilometers":40,"precioViaje":2.40},
  {"id":5,"vehicleType":"furgoneta","date":"2016-08-19T00:00:00.000Z","kilometers":130,"precioViaje":13.00},
  {"id":6,"vehicleType":"camion","date":"2016-09-14T00:00:00.000Z","kilometers":300,"precioViaje":48.00},
  {"id":7,"vehicleType":"camion","date":"2017-02-22T00:00:00.000Z","kilometers":300,"precioViaje":48.00},
  {"id":8,"vehicleType":"moto","date":"2017-04-22T00:00:00.000Z","kilometers":60,"precioViaje":3.60},
  {"id":9,"vehicleType":"camion","date":"2017-06-05T00:00:00.000Z","kilometers":290,"precioViaje":47.00},
  {"id":10,"vehicleType":"furgoneta","date":"2018-03-11T00:00:00.000Z","kilometers":135,"precioViaje":13.75},
  {"id":11,"vehicleType":"moto","date":"2018-05-23T00:00:00.000Z","kilometers":50,"precioViaje":3.50},
  {"id":12,"vehicleType":"camion","date":"2018-07-17T00:00:00.000Z","kilometers":190,"precioViaje":30.40},
  {"id":13,"vehicleType":"camion","date":"2019-02-27T00:00:00.000Z","kilometers":275,"precioViaje":44.00},
  {"id":14,"vehicleType":"moto","date":"2019-04-15T00:00:00.000Z","kilometers":45,"precioViaje":2.70},
  {"id":15,"vehicleType":"camion","date":"2019-05-08T00:00:00.000Z","kilometers":220,"precioViaje":35.20},
  {"id":16,"vehicleType":"camion","date":"2020-03-18T00:00:00.000Z","kilometers":200,"precioViaje":32.00},
  {"id":17,"vehicleType":"furgoneta","date":"2020-10-08T00:00:00.000Z","kilometers":180,"precioViaje":18.90},
  {"id":18,"vehicleType":"moto","date":"2020-12-04T00:00:00.000Z","kilometers":34,"precioViaje":2.04}
]`;

/**
 * Haciendo uso del modelo de datos, guarda en GASTOS_DB un objeto de tipo GastoCombustible por cada
 * registro que exista en el histórico.
 */
export const GASTOS_DB = JSON.parse(jsonHistorico).map(item => {
    return new GastoCombustible(
        item.id,
        item.vehicleType,
        item.date,
        item.kilometers,
        item.precioViaje
    );
});
```

Ejercicio 3: Servicio (gasto.service.js):

- Función almacenarGastos():

```
function almacenarGastos(){
    for (const gasto of GASTOS_DB) {
        // 1. Guardo cada registro en el localStorage usando el ID como clave y el objeto en JSON como valor
        localStorage.setItem(gasto.id, JSON.stringify(gasto));
        console.log("Registro guardado en el localStorage: ", JSON.parse(localStorage.getItem(gasto.id)));

        // 2. Calcular el gasto total acumulado para cada año
        const year = gasto.date.getFullYear(); // Se extrae el año del gasto
        gastoAnual[year] += gasto.precioViaje; // Acumula el gasto anual en el año correspondiente
    }

    // 3. Almaceno el gasto total acumulado de cada año en el sessionStorage
    for (const year in gastoAnual)
        sessionStorage.setItem(year, gastoAnual[year]);
}
```

- Función procesarGasto():

```
function procesarGasto(jsonNuevoGasto){
    // 1. Convierto el JSON recibido a un objeto plano de JavaScript
    const receivedGasto = JSON.parse(jsonNuevoGasto);

    const nuevoGasto = new GastoCombustible(
        receivedGasto.id,
        receivedGasto.vehicleType,
        receivedGasto.date,
        receivedGasto.kilometers,
        receivedGasto.precioViaje
    );

    // en función del año en el que se realizó ese gasto, recupera del sessionStorage
    // el gasto total de ese año, súmale el importe del gasto actual y actualiza el valor almacenado en sessionStorage.

    // Obtengo el año del nuevo gasto
    const year = nuevoGasto.date.getFullYear();

    // Recupero el gasto de ese año desde el sessionStorage (me aseguro de que no es NaN)
    let gastoAnual = sessionStorage.getItem(year);
    if(gastoAnual != null) {
        gastoAnual = parseFloat(gastoAnual);
    } else {
        gastoAnual = 0;
    }

    // Agrego el importe del nuevo viaje al total anual
    gastoAnual += nuevoGasto.precioViaje;

    // Actualizo el valor en el sessionStorage con el nuevo total
    sessionStorage.setItem(year, gastoAnual);

    console.log(`Gasto para el año ${year} actualizado en el sessionStorage. Nuevo gasto almacenado:`, parseFloat(sessionStorage.getItem(year)));
}
```

- Exportación de las funciones a través del objeto GastoService:

```
export const GastoService = {
    almacenarGastos,
    procesarGasto
}
```

Resultados:

Estoy trabajando con Eclipse. Para probar la aplicación web, estoy ejecutando desde la terminal de Eclipse:

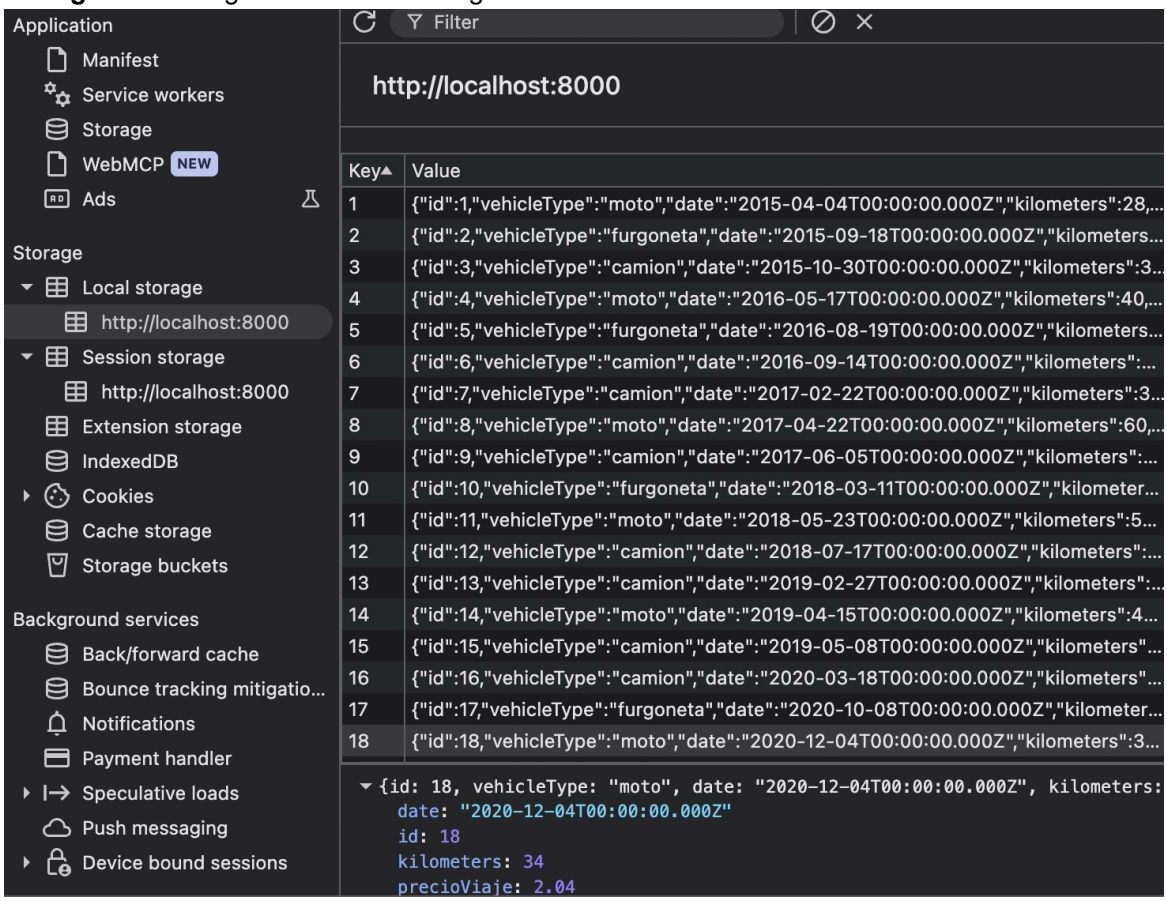
```
python3 -m http.server
```

Una vez terminado el desarrollo, para hacer pruebas en el navegador, he descomentado las líneas del fichero main.js. Además, en dicho fichero, he importado GastoService y ejecutado almacenarGastos():

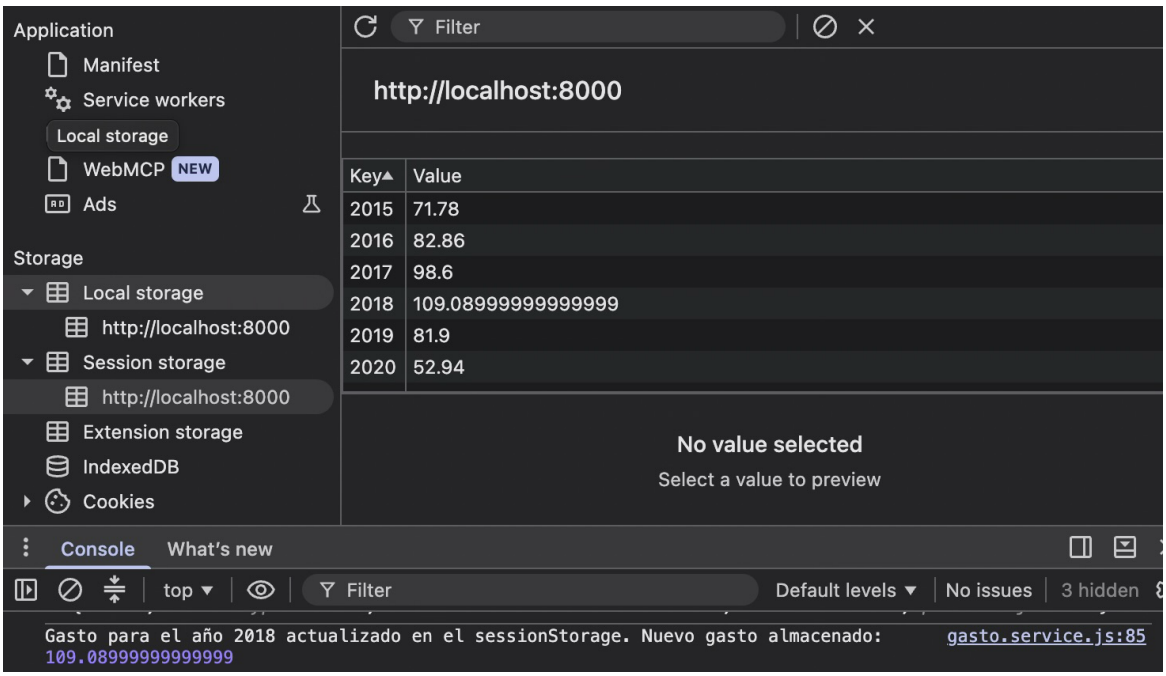
```
import { GastoService } from './services/gasto.service.js';
GastoService.almacenarGastos();
```

Adicionalmente, para poder trabajar mejor, he aumentado el tiempo en que el programa genera un gasto nuevo de 5 segundos a 15.

- **LocalStorage** con los registros del JSON cargados:



- **SessionStorage** almacenando los nuevos registros (tanto los que se autogeneran, como los que se cargan desde la página Web:



4. AUTOEVALUACIÓN

Ejercicio	Apartado	Nota (0-10)	Justificación
1	Estructura de directorios	10	Se ha seguido una estructura de directorios basada en el Modelo VVM.
	Enlazado de documentos	10	Los archivos principales main.js y style.css están correctamente enlazados desde el index.html.
	Separación de responsabilidades	10	Se respeta la separación de responsabilidades y el flujo de llamadas main → service → data → model.
2	Definición del modelo	10	He creado la clase GastoCombustible con todos los atributos que se pedían.
	Importación y parseo de datos	10	Se han importado los registros del JSON (jsonHistorico) y se han convertido en iinstancias de objeto GastoCombustible en la estructura de datos GASTOS_DB.
3	Servicio	10	La función almacenarGastos() almacena los datos en el LocalStorage y en el SessionStorage. Por su parte, la función procesarGasto(), procesa los nuevos registros y actualiza los datos anuales en el SessionStorage.

Nota Final (0-10)	10

5. BIBLIOGRAFÍA

No he utilizado más fuentes bibliográficas además del material proporcionado por el docente en el aula virtual.